ptbr-writer: controlled technical Portuguese for people and agents

Agentic writing in Brazilian Portuguese is a cheap abstraction

Matheus Hoffmann ORCID
Escola Politécnica, Universidade Federal do Rio de Janeiro

Rob Kler Soares da Silva Junior ORCID

October 2026, not peer reviewed · v0.3.0, preprint · rev. 3e36991+122





Abstract. ptbr-writer rewrites technical text in Brazilian Portuguese for a single reading: explicit subject, one proposition per sentence and consistent terms. Its seed is Português Técnico Controlado (PTC), by kayquer: a Claude Code skill with controlled-language rules in Portuguese for a reader that may be a model. ptbr-writer adapts the 8 rules of PTC and adds 5, for 13 rules in total, PTW-1 to PTW-13. Each request goes through one of 5 levels of rigor, from `estrito` to `smart-note`. For text with sources, the `cientifico` skill adds 12 rules, CIE-1 to CIE-12. The same rules reach the writer by three paths: a Claude Code plugin, an installer for OpenCode and a browser extension for Chrome and Firefox. The extension treats a review as a decision. Besides models that generate text (Claude, DeepSeek or a local model through Ollama), it accepts a local decision model, served by Ollaya, which says which rule each sentence breaks, with a probability, and generates no text. The runtime tools are static Rust binaries pinned by sha256, and each claim that was not checked carries the mark ⚠. This paper describes the rules, the levels, the architecture and the evidence policy. It reports no measure of accuracy: the regression cases in `evals/` are written and were not run, and the latency and accuracy of the local models on pt-BR were not measured. The code is under MPL-2.0 since v0.3.0.

Keywords: technical writing; Brazilian Portuguese; controlled language; coding agents; decision models; reproducibility.

1 Introduction

A runbook is read by the person who runs the procedure and, more and more, by an agent that runs it. Neither can stop to ask “did you mean X or Y?”. PTC, by kayquer, wrote this need down as a Claude Code skill, with 8 rules for Brazilian Portuguese [1]. ptbr-writer starts from that seed.

The thesis of this paper is that agentic writing in Brazilian Portuguese is a cheap abstraction. It is cheap because the whole abstraction is text: one file of rules in Markdown, which any agent that reads skills loads, and a small binary that decides what to ask before it writes. There is no trained model, no fine-tuning and no server. The paper supports the thesis with three pillars: tailored Brazilian Portuguese (section 2), portability (section 3) and a harness that improves itself (section 4). Section 5 shows what was measured of cost, and section 6, what was not.

2 Tailored Brazilian Portuguese

PTC notes that English requires the subject, so ASD-STE100, the controlled language of the aerospace industry, never needed a rule for it [1]. Portuguese lets the writer drop the subject. The first rule of ptbr-writer, PTW-1, asks for an explicit subject and avoids the passive `-se`, `o mesmo` ("the same") and ambiguous third-person clitics (`SKILL.md`). A sentence such as "deve-se verificar os logs" ("one must check the logs") becomes "verifique os logs" ("check the logs").

Rules PTW-1 to PTW-8 adapt the 8 of PTC: one proposition per sentence, explicit modality and quantity, full verbs, flat syntax, frozen terms, numbers with units and the spelling of the 1990 Orthographic Agreement, with care for technical prefixes such as `microsserviço`. Rules PTW-9 to PTW-13 add an evidence policy: every claim has a mark, no reference is invented, a number has a unit and a source, there is no priority claim, and fact stays apart from analysis.

The rigor changes with the reader. The levels are `estrito` (runbooks and output for agents), `descritivo` (system documentation), `leve` (announcements), `científico` (text with sources) and `smart-note` (notes for a second brain). The same request can come out in pt-BR and in ASD-STE100, one text per language. Every text the plugin writes carries its reading time, computed by the binary, at 238 words per minute [2]. That rate is the one for English; pt-BR uses the same, with no scaling ⚠.

3 Portability

The rules live in one file, `SKILL.md`, with 807 words (`wc -w`), and the `cientifico` skill has 1,087. An agent that reads skills loads these files with no change. ptbr-writer reaches the writer by three paths:

- **Claude Code:** a plugin, installed with one command and pinned by tag. A repository pins the version in its own `.claude/settings.json`, and everyone who opens it uses the same one.
- **OpenCode:** a script copies the skills as files and installs a plugin that does what the Claude Code hook does. It was tested on OpenCode 1.19.0: the skills load and the instruction reaches the model. That the model answers with the `question` tool was not measured ⚠.
- **Browser:** an extension for Chrome and Firefox 128+ underlines in the text field what the level asks to change. Nothing changes without a click on **Aceitar** (accept).

What runs on the machine is a static Rust binary, `ptbr`, of 570,112 bytes on `linux-x86_64`, with a twin for `linux-aarch64`. `tools.lock` pins the picasso commit that builds it and the sha256 of each binary. There is no Python and no Node at run time. The `.pdf` comes from the machine's pandoc or, without it, from a binary that Nix builds from a pinned commit; the first build, on a fresh GitHub runner with 4 cores, took 892 s.

The extension takes portability to the model. It talks to Claude and DeepSeek, with the user's key, and to two local servers, with no key. Ollama serves a model that writes the replacement. Ollaya serves decision models, which answer closed questions with probabilities and never generate text [3]. For the extension, pointing out a broken rule is a decision: for each sentence, the question is "which rule does this sentence break?", with `nenhuma` (none) and the level's rules as the answers. The two servers also work together: Ollaya picks the tone

of the conversation (concise, Socratic or friendly), and Ollama writes in that tone. When Ollaya is down, Ollama writes alone.

4 A harness that improves itself

In this paper, RSI has one meaning only, the operational one the repository uses: the label `rsi` marks "a mistake the harness let through, whose fix changes the loop so that it does not come back" (`ISSUE_LABELS.md`). The loop is `just quality`, which CI runs on each pull request. Four cases from October 2026 show the pattern, all with the agent itself as the author of the mistake:

1. An edit put two roadmap rows at the top of the README, outside the table. `just quality` passed, because it only checked that each piece's name appeared somewhere. The fix added a structure check: no table row before the first heading.
2. The README reached ~31 minutes of reading. It was cut to ~8, and `just quality` now fails above 10, by the same binary that measures the user's text.
3. The English README showed the figure with Portuguese labels. Each figure got an English twin, and the check fails when a README shows the figure of the other language.
4. The release recipe opened an editor in a shell with no terminal and stopped halfway. Without a terminal, it now does not open the editor and says what to do.

In each case, the fix to the file came with a new check, and the check is what stays. The project does not measure whether this loop reduces mistakes over time ⚠.

5 What was measured of cost

Three real requests to the plugin, with the model `claude-opus-5-5`, at plugin commit `ca0e859`, on 21 October 2026, were measured by `claude -p --output-format json` (`docs/eli5/RESULTS.tsv`):

Request	Level	Tokens in total	Read from cache	Output	Cost (US$)
deploy runbook	estrito	350,529	301,876	8,184	0.5477
announcement	leve	184,688	155,591	2,401	0.2927
explain to a child	descritivo	232,366	200,269	3,434	0.3380

Between 84% and 86% of each request's tokens were read from the cache. The abstraction itself has no training or server cost. The cost per request is that of the model the user picks. With a local decision model, there is no charge per request. On Ollaya Bench, on a 24-core CPU, `laya:multilingual` answered five questions in 515.5 ms and `decima` in 1,212.5 ms, on tasks in English and German (`docs/SURVEY_DECISION_MODEL.md`). These numbers were not measured on pt-BR ⚠.

6 Limitations

The regression cases in `evals/` are written and were not run by a harness, so the accuracy of the rules was not measured. The extension was tested by hand on WhatsApp Web, with a real key; Gmail and GitHub were not tested with the real editor. The latency and accuracy of Ollama and Ollaya on pt-BR were not measured, and Ollaya's 50% threshold was not tuned. The four cases of section 4 are from one project and one month. The three requests of section 5 are a sample of three, with one model. The tag v0.2.0 was never created; v0.3.0 is the first tag since v0.1.0.

7 References

1. kayquer. *Português Técnico Controlado (PTC)*. Claude Code skill, MIT license. https://github.com/kayquer/portugues-tecnico-controlado/tree/cfdffad0ed98cdebfb7f591481889090a4f4e65b, commit `cfdffad`, read on 22 October 2026.
2. Brysbaert, M. (2019). How many words do we read per minute? A review and meta-analysis of reading rate. *Journal of Memory and Language*, 109, 104047. https://doi.org/10.1016/j.jml.2019.104047
3. ollaya-dev. *Ollaya*. Apache-2.0 license. https://github.com/ollaya-dev/ollaya/tree/39619accce0f5836dbeb60d2b14f5a056c2459ce, commit `39619ac`, read on 21 October 2026.

The code, the rules and this text are at https://github.com/h0ffmann/ptbr-writer, under MPL-2.0. The template of this PDF comes from the Paper Fabric of h0ffmann/closed-source, commit `563c339` (MIT).